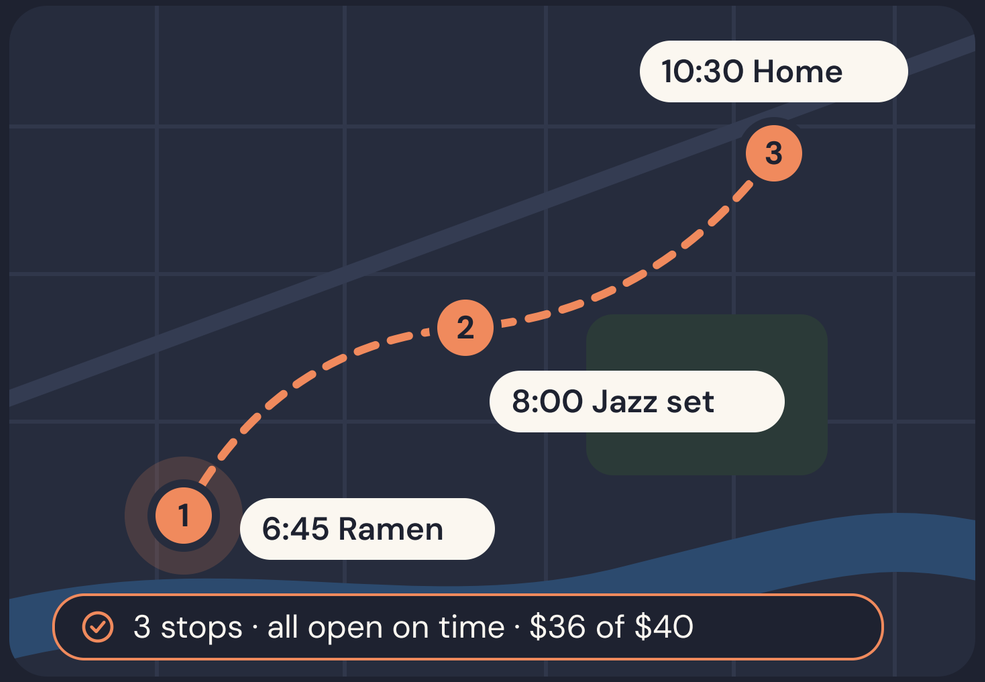
Offhours

An AI agent for free time: live picks for places, concerts and events near you, planned into your calendar.

Who it's for: busy students and young professionals, in any city, with free time in irregular pieces.

Core question: does an LLM paired with a constraint solver plan better than an LLM alone?



Scattered
Ideas live across maps, event sites, feeds and group chats.

Infeasible
Suggestions ignore hours, showtimes, travel time and budget.

Forgotten
Plans miss the calendar, and every search starts from zero.

From one message to a feasible plan

Free from 6:30. Live music under $40, home by 11?

Live search · 3 feasible plans
1. Ramen 6:45 → jazz 8:00
2. Indie show 7:30 → tacos 10:15
3. Comedy 8:00 → dessert

Plan 1, please.

Added to your calendar.

TONIGHT'S PLAN · SOLVED

Ramen, then a jazz set, home by 10:30



- Ramen open from 5 pm
- Jazz at 8:00, 6-min walk
- $36 of a $40 budget
- Home 10:30, before 11

1. **Ask** — Mood, budget, company
2. **Search live** — Places, concerts, events
3. **Solve** — Hours, showtimes, travel
4. **Commit** — Confirm, then calendar
5. **Learn** — Ratings refine picks

The LLM interprets, a solver plans

1 · Request

Free from 6:30. Live music under $40, home by 11?

Plain language in: mood, budget, time, company.

LLM →

2 · Constraints

Window	**6:30–11 pm**
Budget	**≤ $40**
Must	**live music**
Also	**food nearby**
Near	**you, now**

search →

3 · Candidates

- Jazz · 8:00 · $20
- Indie · 7:30 · $25
- Comedy · 8:00 · $15
- Ramen · till 10 pm

+ dozens more records

solver →

4 · Solved plans

1 Ramen → Jazz



2 Indie → Tacos



3 Comedy → Dessert



Orienteering with time windows, in OR-Tools

Then: verify each plan against its sources → LLM explains → you confirm → calendar

Google Calendar · Google Places · Ticketmaster · Weather + travel time · SQLite memory · OR-Tools

Python + FastAPI backend · LLM API with tool calling · React chat UI · recorded API data for evaluation

Any city, one calendar, the full loop

Core build

- Any city; tested in Boston
- Up to 3 stops per plan
- Now + weekend modes
- Google Calendar
- One user, learned preferences

Stretch

- Group plans
- Small-gig web search

Out of scope

- Buying tickets
- Payments
- Mobile app

RISKS AND MITIGATIONS

Invented or stale options

Only search results enter the solver; a verifier checks each plan.

No feasible plan, or a slow solve

Cap stops and candidates; relax soft limits and say what didn't fit.

Thin or costly live data

Blend sources, fall back to evergreen spots, cache and cap API calls.

Calendar writes and location

Confirm every write, with undo; store location only as a neighborhood.

Milestones and how I'll show it works

Oct 29 · Proposal

Scope approved

Feedback on direction before building.

Midpoint

Loop + solver

Chat, live search, solver-built plans, confirmed calendar event.

Final

Benchmark + learning

Hybrid vs. LLM-only results, preference learning in simulation, small pilot.

SUCCESS CRITERIA · 60 SCRIPTED REQUESTS, RECORDED API DATA

≥ 95%

Feasible plans

meet hours, showtimes, travel and budget; reported beside an LLM-only baseline

100%

Grounded

of shown options trace to a live search record

0

Unconfirmed writes

and no duplicate events under injected API failures

< 10 s

Plan latency

median from request to 3 plans, solver included

APPENDIX · SPEAKER NOTES

1 · Problem and audience

Free time is easy to waste on planning. Ideas are spread across apps, and suggestions, including ones from chatbots, often ignore whether a place is open, when a show starts, how long it takes to get there, or what it costs. Even good plans rarely make it onto the calendar. Offhours is a conversational agent that turns “what should we do?” into a feasible, scheduled plan like the one on the map: three stops, each open on time, within budget. Target users: busy students and young professionals, wherever they live. The technical question: does pairing an LLM with a constraint solver produce more feasible plans than an LLM planning on its own?

2 · Proposed solution

The user just talks to the agent. It reads free time from the calendar and runs a live search for places, concerts and events near the user’s location (or where they’ll be) at that time. Then a constraint solver, not the LLM, assembles plans of up to three stops that respect opening hours, showtimes, travel time and budget; the panel on the right shows one solved plan and the checks it passed. The LLM explains the options and handles edits like “something cheaper”. After the user confirms, the plan goes on the calendar with travel buffers, and every accept, skip and rating updates the stored preferences. Two modes: “now” for the next few hours, and weekend planning.

3 · Approach

Walk the request left to right. The LLM does what it is good at: turning a casual request into structured constraints (time window, budget, must-haves like “live music, then food”, location). Live search returns candidate places and events as typed records with hours, showtimes, prices and locations. A constraint solver, OR-Tools, then builds itineraries as an orienteering problem with time windows: choose up to three stops, and their order, to maximize a preference score while respecting opening hours, showtimes, travel time between stops, and budget. A verifier checks every plan against the source records before the user sees it, and the agent re-plans when a check fails. Edits like “swap dinner for something cheaper” become constraint changes and a re-solve. Calendar writes always wait for confirmation.

4 · Scope and risks

To keep this finishable, the core build works in any city that live search covers but is tested first in Boston, with plans of up to three stops, now and weekend modes, Google Calendar, and one user with learned preferences. Group planning and web search for small gigs are stretch goals; buying tickets (the agent links to the seller), payments and a mobile app are out. The main risks: suggestions that don't exist or aren't open, handled by letting only search results into the solver and verifying every plan; requests with no feasible plan, handled by relaxing soft constraints and explaining what didn't fit; thin live data, handled by blending sources and falling back to reliable spots; and calendar writes and location, handled by confirmation, undo and storing only the neighborhood.

5 · Plan and success criteria

After proposal feedback, the midpoint target is the full loop with the solver in it: chat, live search, solver-built plans and a confirmed calendar event. The final adds the benchmark, preference learning and a small usability pilot. The benchmark is 60 scripted requests run against recorded API responses, so every variant sees identical data. The headline result is feasibility: at least 95% of hybrid plans should meet hours, showtimes, travel time and budget, reported next to an LLM-only planner on the same requests. Every option shown must trace to a search record, no calendar write happens without confirmation even under injected API failures, and a median request should return three plans in under ten seconds. Preference learning is evaluated with simulated users who have hidden preferences, since a short pilot can't produce enough data.